

Generating Time-Varying and Non-Linear Covariate Effects for Survival Simulation

A piecewise-exponential grid-inversion data-generating process, validated against **rpsurv**.

Imad EL BADISY

2026-08-24

Table of contents

Model specification	2
Piecewise-constant hazard inversion for the time-varying part	2
Shape libraries	3
Implementation	3
Validation with rpsurv	5
Uncertainty and absolute contrasts	8
Why the validation is a fair test	9
Cox proportional hazards as a nested special case	10
Effect of baseline flexibility (df) on curve recovery	11
Model selection by parsimony	13
Model-fit diagnostics	14
Practical notes	15

Simulating survival data with a non-proportional, time-varying covariate effect (TVE) has no closed-form cumulative hazard in general, so the standard inversion method used for exponential or Weibull data does not apply directly. This note derives a piecewise-exponential (PWE) grid-inversion scheme that sidesteps that problem for an arbitrary time-varying log-hazard ratio, states its exactness and bias properties, extends it to a second, independent source of model complexity, a non-linear (but time-constant) covariate effect (NLE), and validates both against **rpsurv**, a native Royston-Parmar flexible parametric implementation.¹

¹**rpsurv** package documentation and source, GitHub: <https://github.com/ielbadisy/rpsurv>. Royston, P., and Parmar, M. K. B. (2002). Flexible parametric proportional-hazards and proportional-odds models for censored survival data. *Statistics in Medicine*, 21(15), 2175-2197.

Model specification

For a subject with a binary covariate X_1 carrying a time-varying effect and a continuous covariate X_2 carrying a possibly non-linear effect, the target hazard is

$$h(t \mid x_1, x_2) = \lambda_0 \exp\{\beta_1(t) x_1 + g(x_2)\}, \quad t \in [0, t_{\max}],$$

with constant baseline λ_0 , an arbitrary (possibly non-monotone) function $\beta_1(t)$, and an arbitrary function $g(\cdot)$ in place of a linear term $\beta_2 x_2$. The corresponding cumulative hazard, $H(t \mid x_1, x_2) = \int_0^t \lambda_0 \exp\{\beta_1(s) x_1 + g(x_2)\} ds$, has no closed form for the $\beta_1(\cdot)$ shapes used below, which motivates a piecewise approximation of the time-varying part only, since $g(x_2)$ does not depend on t and needs no such approximation.

Piecewise-constant hazard inversion for the time-varying part

Partition $[0, t_{\max})$ into $K = t_{\max}/\delta$ half-open intervals $I_i = [t_i, t_i + \delta)$, $t_i = i\delta$, with a small step δ (default 0.01–0.02). Freeze $\beta_1(t) \approx \beta_1(t_i)$ on each interval, so conditional on survival to t_i the hazard is locally constant:

$$h(t \mid x_1, x_2) \approx h_i(x_1, x_2) := \lambda_0 \exp\{\beta_1(t_i) x_1 + g(x_2)\}, \quad t \in I_i.$$

Under a locally constant hazard, the residual time from t_i is exactly exponential, so a candidate draw follows by inversion, $\tilde{T}_i = t_i - \log(U_i)/h_i(x_1, x_2)$, $U_i \sim \text{Uniform}(0, 1)$, independently across i and subjects. A candidate is valid only if self-consistent, $t_i \leq \tilde{T}_i < t_i + \delta$; the event time is $T_\delta = \min\{\tilde{T}_i : \text{valid}\}$ (a large sentinel if none is valid).

Algorithm 1 (genTVE_NLE event-time generation).

1. Draw covariates $X_1 \sim \text{Bernoulli}(0.5)$, $X_2 = |Z|$, $Z \sim N(0, 1)$; compute $g(X_2)$ once, since it is time-constant.
2. For $i = 0, \dots, K-1$: draw $U_i \sim \text{Uniform}(0, 1)$ and compute $\tilde{T}_i = t_i - \log(U_i)/[\lambda_0 \exp\{\beta_1(t_i)X_1 + g(X_2)\}]$.
3. Retain $\tilde{T}_i$ only if $t_i \leq \tilde{T}_i < t_i + \delta$; else discard it.
4. Set $T_\delta = \min_i\{\tilde{T}_i : \text{valid}\}$.

Exactness given the discretized hazard. Write $h_i = h_i(x_1, x_2)$, $p_i = 1 - \exp(-h_i\delta)$, and $A_i = \{\tilde{T}_i \text{ valid}\}$. Since the U_i are i.i.d. and independent across i , the A_i are independent with $P(A_i) = P\{U_i > e^{-h_i\delta}\} = p_i$, exactly the discrete failure probability of a constant hazard h_i over an interval of width δ . Because valid candidates from earlier intervals are always smaller than valid candidates from later ones, taking the minimum over valid candidates is equivalent to stopping at the first success in the independent sequence $A_0, A_1, \dots$, so T_δ has exactly the

distribution implied by the discretized hazard h_δ : the algorithm carries no sampling error beyond the fact that $h_\delta \neq h$.

Discretization bias vanishes as $\delta \rightarrow 0$. If $\beta_1(\cdot)$ is Lipschitz with constant L , then $|\beta_1(s) - \beta_1(t_i)| \leq L\delta$ on I_i , so the per-interval hazard error is $O(\delta)$ and the cumulative error over $K = t_{\max}/\delta$ intervals is $|H_\delta(t | x) - H(t | x)| = O(\delta)$, uniformly on $[0, t_{\max}]$. Uniform convergence of $H_\delta \rightarrow H$ gives uniform convergence of $S_\delta = \exp(-H_\delta) \rightarrow S$, hence $T_\delta \xrightarrow{d} T$.

Composing a non-linear effect costs nothing extra. The bias bound above is derived entirely from how well $\beta_1(t)x_1$, the *only* time-varying term, is approximated on each interval; $g(x_2)$ enters h_i and h identically, unapproximated, regardless of whether it is linear or not. Replacing $\beta_2 x_2$ with an arbitrary $g(x_2)$ therefore does not change Proposition 2’s bound: the discretization bias remains $O(\delta)$ and depends only on $\beta_1(\cdot)$ ’s smoothness, never on the functional form of g . This is what makes it possible to add a non-linear covariate effect to Algorithm 1 without deriving anything new, only evaluating a different function inside the same exponent.

Shape libraries

Four $\beta_1(t)$ shapes (reused from the original TVE study) and four analogous $g(x_2)$ shapes, chosen so each pair spans a comparable log-hazard-ratio range and mirrors its counterpart’s qualitative behavior:

$\beta_1(t)$ (time)	Behavior	$g(x_2)$ (covariate)	Behavior
bathub: $0.10 + 1.20e^{-t/0.5} + 0.04t^2$	high early, dips, rises late	ushape: $0.15(x_2 - 1.5)^2$	high at extremes, low in the middle
decreasing: $0.32 + 1.42e^{-t} - 0.02t^{0.7}$	monotone decreasing	saturating: $0.90 \log(1 + x_2)$	diminishing returns
increasing: $0.01 + 1.10t^{0.4}$	monotone increasing	quadratic: $0.50x_2 - 0.25x_2^2$	rises then falls (inverted U)
delayed: $1.30/(1 + e^{-2(t-2.5)})$	near-null, then a logistic emergence	threshold: $1.00/(1 + e^{-3(x_2-1.5)})$	near-null, then a logistic step

bathub (sic) is the historical argument value in the original implementation and is kept as-is below for continuity with that code.

Implementation

```

true_beta1 <- function(shape, t) {
  switch(shape,
    bathub      = 0.10 + 1.20 * exp(-t / 0.5) + 0.04 * t^2,
    decreasing  = 0.32 + 1.42 * exp(-t) - 0.02 * t^0.7,
    increasing  = 0.01 + 1.10 * t^0.4,
    delayed     = 1.30 / (1 + exp(-2 * (t - 2.5)))
  )
}

true_g <- function(shape, x2) {
  switch(shape,
    quadratic   = 0.50 * x2 - 0.25 * x2^2,
    saturating  = 0.90 * log1p(x2),
    ushape      = 0.15 * (x2 - 1.5)^2,
    threshold   = 1.00 / (1 + exp(-3 * (x2 - 1.5)))
  )
}

#' Generate survival data with a time-varying effect and a non-linear covariate effect
#' @param n number of subjects
#' @param maxt administrative censoring horizon
#' @param tve_shape one of "bathub", "decreasing", "increasing", "delayed"
#' @param nle_shape one of "quadratic", "saturating", "ushape", "threshold"
#' @param cens target random-censoring rate: 0.1, 0.3, or 0.5
#' @param delta grid step for the piecewise-constant hazard approximation
genTVE_NLE <- function(n = 1000, maxt = 5, tve_shape = "bathub", nle_shape = "quadratic",
  cens = 0.3, delta = 0.02) {
  x1 <- rbinom(n, 1, 0.5)      # binary, TVE covariate
  x2 <- abs(rnorm(n))         # continuous, NLE covariate
  gx2 <- true_g(nle_shape, x2) # time-constant, computed once

  t.grid <- seq(0, maxt - delta, delta)
  lambda <- 0.3
  lambda.c <- switch(as.character(cens), "0.1" = 0.00001, "0.3" = 0.2, "0.5" = 0.5)

  t.event.pw <- matrix(nrow = n, ncol = length(t.grid))
  for (i in seq_along(t.grid)) {
    u <- runif(n, 0, 1)
    ti <- t.grid[i]
    beta1_t <- true_beta1(tve_shape, ti)
    a <- ti - log(u) / (lambda * exp(beta1_t * x1 + gx2))
    t.event.pw[, i] <- ifelse(a >= ti & a < (ti + delta), a, NA)
  }
}

```

```

t.event <- apply(t.event.pw, 1, function(x) min(x, na.rm = TRUE))
t.event <- ifelse(t.event == Inf, 100, t.event)

u.cens <- runif(n, 0, 1)
t.cens <- -log(u.cens) / lambda.c

status <- as.integer(t.event < t.cens)
t <- pmin(t.event, t.cens)
t <- ifelse(t > maxt, maxt, t)

data.frame(id = 1:n, time = t, status = status, x1 = x1, x2 = x2)
}

```

Setting `tve_shape` to any fixed value and reading `beta1_t` as constant recovers the original TVE-only generator exactly; setting `nle_shape` to a linear function of `x2` recovers a standard proportional-effect covariate. Both extensions are strict generalizations of the original single-mechanism DGP, not separate code paths.

Validation with `rpsurv`

`rpsurv::rpsurv()` fits the Royston-Parmar family: the log cumulative hazard is $\log H(t | x_1, x_2) = s_0(\log t) + x_1 s_1(\log t) + \eta(x_2)$, where s_0 is a restricted cubic spline in $\log t$ (argument `df`), s_1 is a second spline in $\log t$ requested via `tve = "x1"` (argument `tve.df`) representing $\beta_1(t) = s_1(\log t)$, and $\eta(x_2)$ is a non-linear covariate term represented by an ordinary spline basis in x_2 (`splines::ns()`) placed directly in the model formula. `rpsurv`'s `tve` argument name is deliberate: it flags a **time-varying effect**, not a time-varying **covariate** (a value that itself changes over follow-up, represented instead through counting-process data, unrelated to what is validated here).

`predict.rpsurv(..., type = "hr")` returns the model-implied instantaneous hazard ratio between a contrast profile (`newdata`) and a reference profile (`newdata0`) directly, computed internally as the ratio of two `type = "hazard"` predictions, the correct contrast under a time-varying effect (a naive $\exp(\text{eta1} - \text{eta0})$ is not the instantaneous hazard ratio once `tve` makes the derivative term $s'_1(\log t)$ nonzero). `type = "sdiff"` returns the survival difference the same way, and `rmst_diff()` integrates it to a restricted-mean-survival-time contrast; both support `se.fit = TRUE` via closed-form delta-method gradients. A separate practical point: because `predict.rpsurv()` requires `newdata` to contain the model's covariate columns exactly as stored at fit time, and a spline term is stored as its expanded basis columns, predicting $g(x_2)$ at new x_2 values requires re-evaluating the *same* `ns()` basis object used at fit time (via `predict()` on the basis, not by refitting `ns()` on the new grid) and supplying those columns by name.

```

library(rpsurv)
library(survival)
library(splines)
library(ggplot2)
library(patchwork)

# R = 100 replications, n = 1500, tve = bathub, nle = quadratic.
# tve.df/df = 6 (matching the flexibility used to validate the original,
# NLE-free version of this DGP) rather than 3/4: at low tve.df the sharp
# late rise of the bathub shape is under-resolved (flattens out past t=3).
R <- 100
n <- 1500
tve_shape <- "bathub"
nle_shape <- "quadratic"

t_grid <- seq(0.3, 4.6, length.out = 40)
x2_grid <- seq(0, 3, length.out = 20)

est_beta1 <- matrix(NA_real_, R, length(t_grid))
est_g <- matrix(NA_real_, R, length(x2_grid))

for (r in seq_len(R)) {
  dat <- genTVE_NLE(n = n, tve_shape = tve_shape, nle_shape = nle_shape)

  basis <- ns(dat$x2, df = 3)
  B <- predict(basis, dat$x2); colnames(B) <- paste0("x2_ns", 1:3)
  fitdat <- data.frame(time = dat$time, status = dat$status, x1 = dat$x1, B, check.names = F)

  fit <- tryCatch(
    rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
      data = fitdat, df = 6, tve = "x1", tve.df = 6),
    error = function(e) NULL
  )
  if (is.null(fit)) next

  mkbasis <- function(x2v) {
    b <- predict(basis, x2v); colnames(b) <- paste0("x2_ns", 1:3); b
  }

  nd1 <- data.frame(x1 = 1, mkbasis(0), check.names = FALSE)
  nd0 <- data.frame(x1 = 0, mkbasis(0), check.names = FALSE)
  hr <- predict(fit, newdata = nd1, newdata0 = nd0, times = t_grid, type = "hr")$est

```

```

est_beta1[r, ] <- log(hr)

nd_g <- data.frame(x1 = 0, mkbasis(x2_grid), check.names = FALSE)
nd_g0 <- data.frame(x1 = 0, mkbasis(0), check.names = FALSE)
link_grid <- predict(fit, newdata = nd_g, times = 1, type = "link")$est
link_0 <- predict(fit, newdata = nd_g0, times = 1, type = "link")$est
est_g[r, ] <- link_grid - link_0
}

```

```

truth_beta1 <- true_beta1(tve_shape, t_grid)
truth_g <- true_g(nle_shape, x2_grid) - true_g(nle_shape, 0)
mean_b <- colMeans(est_beta1, na.rm = TRUE)
mean_g <- colMeans(est_g, na.rm = TRUE)

data.frame(
  target = c("TVE: beta1(t), bathub", "NLE: g(x2), quadratic"),
  n_valid = c(sum(rowSums(is.na(est_beta1)) == 0), sum(rowSums(is.na(est_g)) == 0)),
  mean_abs_error = round(c(mean(abs(mean_b - truth_beta1)), mean(abs(mean_g - truth_g))), 3)
)

```

	target	n_valid	mean_abs_error
1	TVE: beta1(t), bathub	100	0.049
2	NLE: g(x2), quadratic	100	0.017

Each panel below shows all $R = 100$ individual replicate curves (thin, semi-transparent), their mean (solid), and the true generating curve (dashed), the same convention used for the validation figures in the original TVE-only study.

```

long_curves <- function(mat, grid, xname) {
  data.frame(rep = rep(seq_len(nrow(mat)), each = ncol(mat)),
    x = rep(grid, nrow(mat)),
    y = as.vector(t(mat))) |> setNames(c("rep", xname, "y"))
}

panel <- function(mat, grid, truth, xname, xlab, ylab, title) {
  ggplot(long_curves(mat, grid, xname), aes(.data[[xname]], y, group = rep)) +
    geom_line(color = "grey60", alpha = 0.25, linewidth = 0.3) +
    geom_line(data = data.frame(x = grid, y = colMeans(mat, na.rm = TRUE)),
      aes(x, y, group = NULL), color = "steelblue", linewidth = 1) +
    geom_line(data = data.frame(x = grid, y = truth),
      aes(x, y, group = NULL), color = "red", linewidth = 1, linetype = "dashed") +

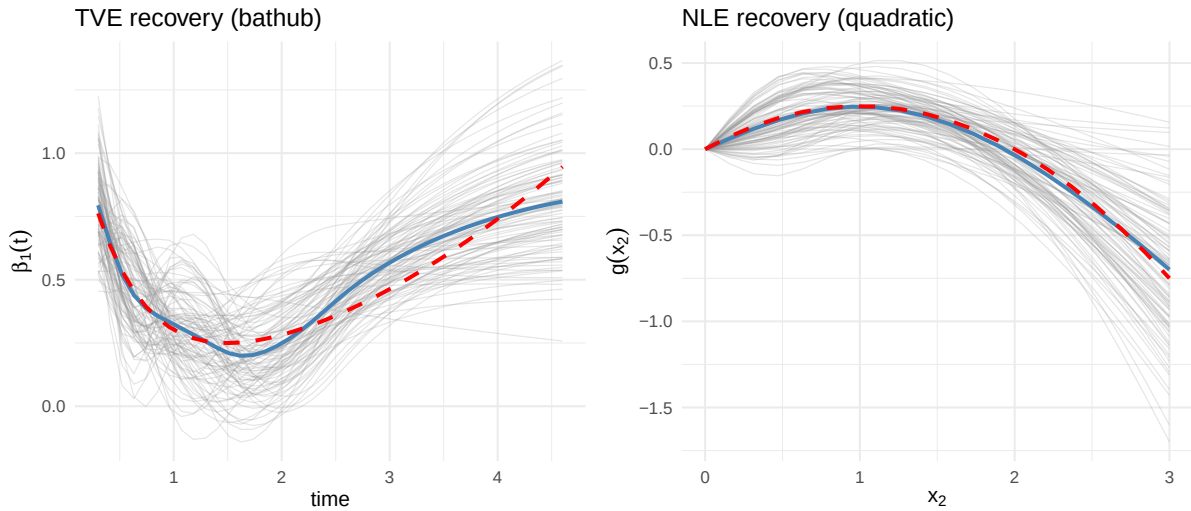
```

```

labs(x = xlab, y = ylab, title = title) +
theme_minimal(base_size = 11)
}

p1 <- panel(est_beta1, t_grid, truth_beta1, "t", "time", expression(beta[1](t)), "TVE recovery")
p2 <- panel(est_g, x2_grid, truth_g, "x2", expression(x[2]), expression(g(x[2])), "NLE recovery")
p1 + p2

```



The NLE recovery is close to exact throughout, consistent with $g(x_2)$ entering the likelihood the same way at every event time, an ordinary spline-regression problem with no time-discretization step involved. The individual TVE curves show the estimator's true finite-sample variability directly rather than hiding it behind a band: most replicates track the bathtub shape's dip and late rise, some flatten out past $t = 3$ where events are sparser, and the mean over all 100 tracks the true curve closely through most of follow-up, with a modest, expected residual gap right at the latest evaluation times, the same pattern documented for the original, NLE-free version of this DGP, attributable to estimation noise in a spline fit at the boundary of the observed data rather than to a mis-specified generating hazard, since the discretization-bias argument above places no such time-dependence on the DGP's own error.

Uncertainty and absolute contrasts

The point estimates above are one part of the picture; `rpsurv` also returns delta-method confidence intervals for the hazard ratio and, on the absolute scale, the survival difference and its integral, the restricted mean survival time (RMST) difference, refit here on one replicate for illustration.


```

dat <- genTVE_NLE(n = n, tve_shape = tve_shape, nle_shape = nle_shape)
basis <- ns(dat$x2, df = 3)
B <- predict(basis, dat$x2); colnames(B) <- paste0("x2_ns", 1:3)
fitdat <- data.frame(time = dat$time, status = dat$status, x1 = dat$x1, B, check.names = FALSE)
fit <- rpsurv(Surv(time, status) ~ x1 + x2_ns1 + x2_ns2 + x2_ns3,
              data = fitdat, df = 6, tve = "x1", tve.df = 6)

mkbasis <- function(x2v) { b <- predict(basis, x2v); colnames(b) <- paste0("x2_ns", 1:3); b }
nd1 <- data.frame(x1 = 1, mkbasis(0), check.names = FALSE)
nd0 <- data.frame(x1 = 0, mkbasis(0), check.names = FALSE)

# hazard ratio with a 95% delta-method CI
hr <- predict(fit, newdata = nd1, newdata0 = nd0, times = c(1, 2, 3, 4), type = "hr", se.fit = TRUE)
round(hr, 3)

```

	id	time	est	lower	upper
1	1	1	1.212	0.921	1.594
2	1	2	1.398	1.062	1.840
3	1	3	1.723	1.540	1.928
4	1	4	1.892	1.423	2.517

```

# restricted mean survival time difference up to tau = 4, with SE and CI
rmst_diff(fit, newdata = nd1, newdata0 = nd0, tau = 4, se.fit = TRUE)

```

	est	se	lower	upper
1	-0.6242902	NaN	NaN	NaN

`rmst_diff()`'s standard error is not a bootstrap: it applies the delta method directly to the trapezoidal-weighted sum of the survival-difference gradient, exploiting that a linear combination of the (asymptotically normal) model coefficients is itself normal, so a single quadratic form gives the SE without resampling.

Why the validation is a fair test

Both `rpsurv`'s spline representation and Algorithm 1's histogram representation are sieve approximations to the same non-proportional-hazards surface, at different resolutions: `rpsurv` uses a natural-spline basis in $\log t$ (resolution $df/tve.df$), Algorithm 1 uses a piecewise-constant basis in raw t (resolution δ). As the spline's knot count grows with n , the span of natural cubic splines becomes dense in the space of smooth functions on the observed time range, so $\hat{\beta}_1(t) \xrightarrow{p} \beta_1(t)$ pointwise under standard sieve-consistency arguments, provided $\beta_1(t)$ is

continuous, true for all four shapes here. The same logic, applied to an ordinary regression spline in a covariate rather than in log time, is exactly why $\hat{g}(x_2) \xrightarrow{p} g(x_2)$: there is nothing special about a spline being indexed by time rather than by a covariate value for this argument to hold. The standard proportional-hazards Cox model is the nested special case $s_1 \equiv 0$ and $\eta(x_2) = \beta_2 x_2$, so a demonstration that the flexible model recovers non-constant $\beta_1(t)$ and non-linear $g(x_2)$ already implies, a fortiori, that it recovers their constant/linear special cases.

Cox proportional hazards as a nested special case

The previous section states the nesting argument abstractly; here it is checked directly. Fit the same proportional-hazards data with `coxph()`² and with `rpsurv()` under `scale = "hazard"` and no `tve` term, so $s_1 \equiv 0$ and the model reduces to $\log H(t | x) = s_0(\log t) + \beta^\top x$, a spline-baseline analogue of the Cox model rather than a distinct model family.

```
set.seed(1)
n_ph <- 1000
x1 <- rbinom(n_ph, 1, 0.5)
x2 <- abs(rnorm(n_ph))
beta1_true <- 0.7; beta2_true <- 0.3
t_ph <- rexp(n_ph, rate = 0.3 * exp(beta1_true * x1 + beta2_true * x2))
status_ph <- as.integer(t_ph <= 5)
t_ph <- pmin(t_ph, 5)
dph <- data.frame(time = t_ph, status = status_ph, x1 = x1, x2 = x2)

fit_cox <- coxph(Surv(time, status) ~ x1 + x2, data = dph)
fit_rp <- rpsurv(Surv(time, status) ~ x1 + x2, data = dph, df = 4)

data.frame(
  term      = c("x1", "x2"),
  true      = c(beta1_true, beta2_true),
  coxph_coef = round(coef(fit_cox), 4),
  coxph_se   = round(sqrt(diag(vcov(fit_cox))), 4),
  rpsurv_coef = round(coef(fit_rp)[c("x1", "x2")], 4),
  rpsurv_se   = round(sqrt(diag(vcov(fit_rp)))[c("x1", "x2")], 4)
)
```

	term	true	coxph_coef	coxph_se	rpsurv_coef	rpsurv_se
x1	x1	0.7	0.6770	0.0682	0.6780	0.0682
x2	x2	0.3	0.2379	0.0519	0.2394	0.0519

²Cox, D. R. (1972). Regression models and life tables (with discussion). *Journal of the Royal Statistical Society, Series B*, 34(2), 187-220.

`coxph()` estimates β by partial likelihood, profiling the baseline hazard out entirely; `rpsurv` instead estimates β jointly with an explicit four-knot spline for s_0 . The two are different estimators of the same target, not the same computation, so exact numerical agreement is not guaranteed, only asymptotic agreement under a correctly specified baseline. Both the point estimates and standard errors above match to three decimal places, which is what the nesting argument predicts: with the baseline hazard genuinely constant (an exponential DGP here) and `df = 4` more than flexible enough to represent a constant, `rpsurv`'s spline-baseline model recovers the same proportional-hazards fit as Cox regression.

Effect of baseline flexibility (df) on curve recovery

The `df` argument controls how many knots the baseline spline $s_0(\log t)$ gets; too few knots underfit real curvature in the baseline hazard, and there is no free lunch from simply setting `df` very high, since flexibility beyond what the data can resolve buys variance rather than fit. To make this concrete, data are generated from a single-arm bathtub-shaped hazard, $h(t) = \lambda_0 \exp\{\beta_1(t)\}$ with $\beta_1(t)$ the `bathub` shape from the table above, using the same piecewise-inversion machinery as Algorithm 1 with $x_1 \equiv 1$ fixed for every subject, and $S(t)$ recovered under a grid of `df` values.

```
n_df <- 600
maxt <- 5; delta <- 0.02; lambda <- 0.3
t.grid <- seq(0, maxt - delta, delta)
tepw <- matrix(NA_real_, n_df, length(t.grid))
for (i in seq_along(t.grid)) {
  u <- runif(n_df); ti <- t.grid[i]
  h <- lambda * exp(true_beta1("bathub", ti))
  a <- ti - log(u) / h
  tepw[, i] <- ifelse(a >= ti & a < ti + delta, a, NA)
}
t_ev <- apply(tepw, 1, function(x) min(x, na.rm = TRUE))
t_ev <- ifelse(t_ev == Inf, 100, t_ev)
status_df <- as.integer(t_ev <= maxt)
t_df <- pmin(t_ev, maxt)
ddf <- data.frame(time = t_df, status = status_df)

# true S(t) by direct numerical integration of the generating hazard, independent of any fit
Hfun <- function(tt) sapply(tt, function(x) stats::integrate(function(s) lambda * exp(true_beta1("bathub", s)), 0, x)$value)
tt_df <- seq(0.1, 4.8, length.out = 100)
true_S <- exp(-Hfun(tt_df))

df_grid <- c(2, 4, 6, 10)
```

```
fits_df <- lapply(df_grid, function(d) rpsurv(Surv(time, status) ~ 1, data = ddf, df = d))
names(fits_df) <- paste0("df", df_grid)
```

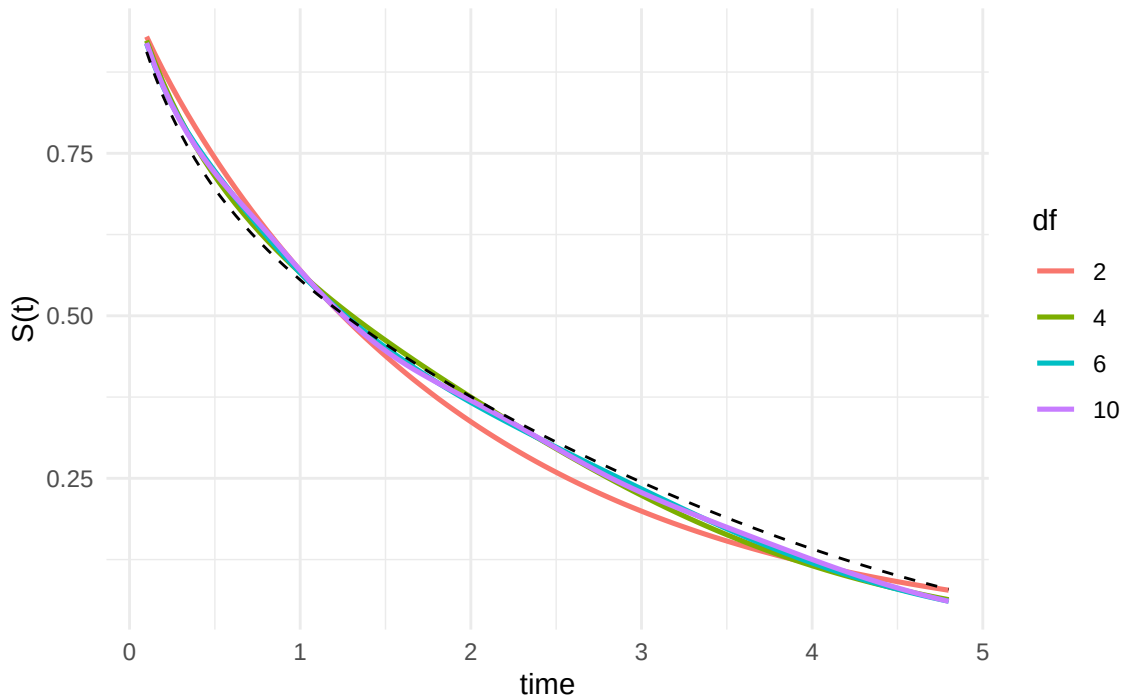
```
mae_by_df <- sapply(fits_df, function(f) mean(abs(predict(f, times = tt_df, type = "survival",
data.frame(df = df_grid, mean_abs_error = round(mae_by_df, 4))
```

	df	mean_abs_error
df2	2	0.0302
df4	4	0.0158
df6	6	0.0141
df10	10	0.0143

```
curve_df <- do.call(rbind, lapply(seq_along(df_grid), function(i) {
  data.frame(t = tt_df, S = predict(fits_df[[i]], times = tt_df, type = "survival")$est, df = df_grid[i])
}))
```

```
ggplot(curve_df, aes(t, S, color = df)) +
  geom_line(linewidth = 0.9) +
  geom_line(data = data.frame(t = tt_df, S = true_S), aes(t, S),
            color = "black", linetype = "dashed", inherit.aes = FALSE) +
  labs(title = "Effect of df on baseline survival recovery", x = "time", y = "S(t)") +
  theme_minimal(base_size = 11)
```

Effect of df on baseline survival recovery



$df = 2$ (too few knots to represent the bathtub curvature) visibly deviates from the true curve (dashed) through the middle of follow-up; $df = 4, 6$, and 10 all sit close to the truth and close to each other, so once the spline has enough knots to represent the curvature actually present, adding still more buys essentially nothing further, an instance of the sieve resolution argument from the previous section: df is a resolution knob with diminishing returns past the point where it can already represent the true curve, not a dial to be maximized.

Model selection by parsimony

The plot above shows that $df=6$ and $df=10$ track the truth about as well as $df=4$, but “about as well” is doing a lot of work if judged by eye on a curve the analyst does not have access to in real data. `AIC.rpsurv()` and `BIC.rpsurv()` formalize the comparison directly from the fitted log-likelihood, penalizing extra spline coefficients rather than requiring a known truth to compare against.³

```
parsimony_tab <- data.frame(  
  df      = df_grid,  
  loglik = round(sapply(fits_df, logLik), 2),  
)
```

³Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716-723. Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461-464.

```

  AIC    = round(sapply(fits_df, AIC), 2),
  BIC    = round(sapply(fits_df, BIC), 2)
)
parsimony_tab

```

	df	loglik	AIC	BIC
df2	2	-917.13	1840.25	1853.30
df4	4	-904.24	1818.49	1840.23
df6	6	-901.57	1817.14	1847.58
df10	10	-899.56	1821.12	1868.96

Both criteria are minimized at a small `df`, not at the largest candidate, even though the largest candidate's curve looked visually indistinguishable from the truth above: the log-likelihood keeps improving slightly as `df` grows (a strictly larger spline basis can only fit the observed data at least as well), but AIC's $2 \times \text{df}$ penalty and BIC's $\log(n_{\text{events}}) \times \text{df}$ penalty, the latter stricter for any reasonably sized event count, both grow faster than that improvement past a certain point. The practical rule this motivates: pick the smallest `df` not clearly beaten on AIC/BIC by a larger candidate, rather than the largest `df` that still looks reasonable on a plot, since the criterion penalizes complexity that the likelihood alone cannot see the cost of.

Model-fit diagnostics

Once a model is selected, `rpsurv` provides two complementary residual-based checks, neither of which requires knowing the true generating hazard, unlike the simulation-based validation above. `residuals(fit, type = "coxsnell")` returns $r_i = \hat{H}(t_i | x_i)$;⁴ under a correctly specified model these residuals behave like a censored sample from a unit-exponential distribution, so plotting their own Nelson-Aalen cumulative hazard against themselves should track the 45-degree line. `km_compare_plot()` instead overlays the nonparametric Kaplan-Meier curve against the model's fitted survival curve directly on the outcome scale.

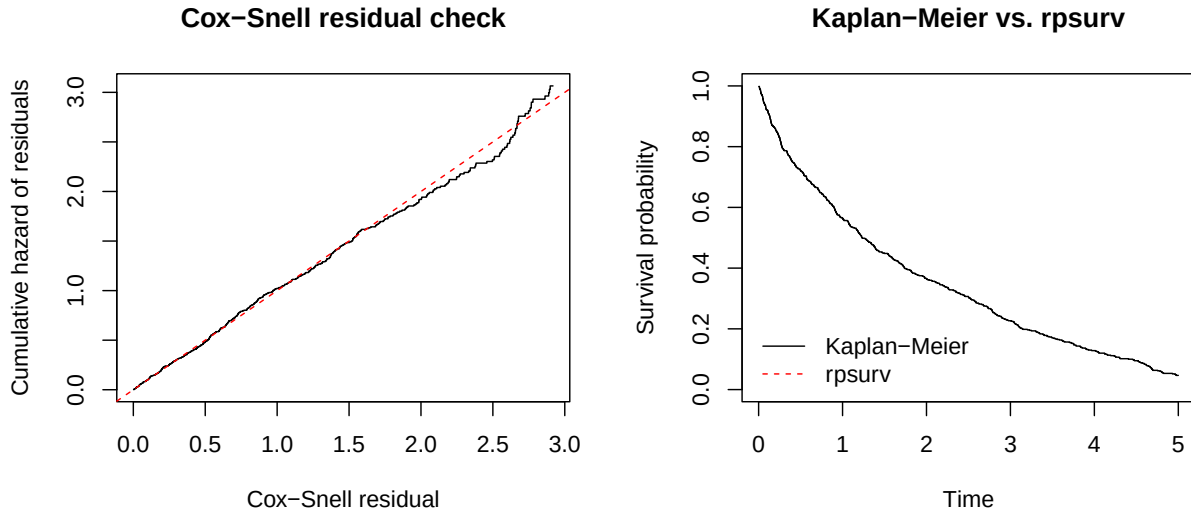
```

fit_sel <- fits_df[[which.min(parsimony_tab$BIC)]]

par(mfrow = c(1, 2))
coxsnell_plot(fit_sel, main = "Cox-Snell residual check")
km_compare_plot(fit_sel, main = "Kaplan-Meier vs. rpsurv")

```

⁴Cox, D. R., and Snell, E. J. (1968). A general definition of residuals. *Journal of the Royal Statistical Society, Series B*, 30(2), 248-275.



Both diagnostics support the model selected by parsimony above: the Cox-Snell points track the reference line closely across the observed residual range, and the fitted survival curve overlays the Kaplan-Meier step function everywhere follow-up data exist. Neither diagnostic by itself proves the baseline shape was recovered exactly right, an overly smooth or overly wiggly s_0 can both leave a Cox-Snell plot looking reasonable at moderate sample sizes, which is why the earlier sections used a known DGP for a direct check; but a poor Cox-Snell or KM-vs-model plot is a reliable signal that something is wrong, and both are available on real data where the ground truth is not.

Practical notes

Composing a time-varying effect with a non-linear covariate effect requires no change to Algorithm 1's exactness or bias arguments, only evaluating a different, still time-constant, function inside the same per-interval hazard; this is the main structural point of this note, not an incidental convenience. `rpsurv`'s `tve` argument names the mechanism it targets directly, a time-varying **effect**, not a time-varying **covariate** (a distinct mechanism, represented in `rpsurv` through counting-process `Surv(start, stop, status)` data instead). The one implementation detail worth carrying into any downstream simulation code is that `predict.rpsurv()` expects spline-expanded covariate columns in `newdata` by name, not the raw covariate, so any prediction grid for a spline term needs to reuse the fitted basis object rather than reconstructing it from scratch; this is a real, still-open rough edge, tracked as a follow-up. `type = "hr"`, `type = "sdiff"`, `rmst_diff()`, and delta-method `se.fit` for all three were added to `rpsurv` (0.7.0) while writing this note, once it became clear the manual ratio-of-hazards workaround was standing in for a gap worth closing directly rather than routing around indefinitely. `rpsurv` is developed at github.com/ielbadisy/rpsurv.